4.7. Behavior

So far the objects presented have contained only data. When you created an object to hold data about page views, you also created a helper function that acted on the data structure. If a function is going to change an object, there's a very good chance that it should belong to the object. In general, objects should be responsible for their own data. This means the `views` object should own the data it contains and be responsible for manipulating it. As you've learned, a function is called a *method* when it's called against an object. The methods of an object provide the behavior of that object.

In this section you'll learn that creating objects from a prototype can lead to more elegant code. First, consider the page views program from section 4.3.2 and what will happen when you need to extend it.

4.7.1. Refactor

In the first page views program (listing 4.5), there's a single `views` object. Both the `increment` function and the `total` function act on the `views` object by knowing which variable it's assigned to. If you wanted to track views for two different websites, then you could manually create two `views` objects:

```
businessViews = {}
pleasureViews = {}
```

Then you could change `increment` and `total` so that the object they're acting on is passed in as an argument:

```
increment = (recipient, key) ->
  recipient[key] ?= 0
  recipient[key] = recipient[key] + 1

total = (recipient) ->
  sum = 0
  for own page of recipient
    sum = sum + 1
  sum
```

The recipient argument is the object to use.